

Microservices Architecture

Complete Analytics & API Reference

Project: E-commerce platform for Indian textiles (Paithani sarees)

Stack: Spring Boot (Java 21) + Next.js (TypeScript) + Node.js

Services: 11 backend microservices + 2 Next.js frontends

Database: PostgreSQL (per service) + RabbitMQ + AWS S3

Auth: RSA JWT (asymmetric keys) + OAuth (Google/Facebook)

1. Service Overview

All backend services with ports, databases and responsibility.

Service	Port	Database	Tech	Responsibility
server_registry	8761	None	Spring Eureka	Service discovery — all services register here
authentication	8081	PostgreSQL gajraj_paithani_auth	Spring Boot	JWT auth, OAuth, refresh tokens, user management
customer	8082	PostgreSQL gajraj_paithani_customers	Spring Boot	Customer profiles, addresses
order	8083	PostgreSQL gajraj_paithani_orders	Spring Boot	Order lifecycle, status tracking
worker	8084	PostgreSQL gajraj_paithani_workers	Spring Boot	Worker profiles, assignments, performance, verification
manager	8085	PostgreSQL gajraj_paithani_manager	Spring Boot	Order flow, reports, support, price changes
owner	8086	PostgreSQL gajraj_paithani_owner	Spring Boot	Owner dashboard, approve/reject reports & prices
product	8087	PostgreSQL gajraj_paithani_products + S3	Spring Boot	Products, variants, inventory, cart, wishlist, images
payment	8088	PostgreSQL gajraj_paithani_payments	Spring Boot	Razorpay payment create & verify
shipping	8089	PostgreSQL gajraj_paithani_shipping	Spring Boot	Shipment create, tracking, cancellation
notification	4000	None	Node.js	Email notifications via RabbitMQ consumer
customer-frontend	3000	None	Next.js	Customer storefront — products, cart, orders, payment
manager-frontend	3002	None	Next.js	Internal dashboard — orders, workers, inventory

2. Service Communication (Feign Clients)

Inter-service HTTP calls using OpenFeign via Eureka service discovery.

Service	Calls To	Feign Interface	Why
authentication	customer	ConnectionInterfaceForCustomer	Save new customer record on register
authentication	manager	ConnectionInterfaceForManager	Save new manager record on register
authentication	worker	ConnectionInterfaceForWorker	Save new worker record on register
customer	authentication	ConnectionInterface	Get user info
customer	product	ProductConnectionInterface	Access product data

order	product	ProductServiceClient	Decrement stock on order create
order	manager	ManagerServiceClient	Create order flow entry in manager
owner	manager	ManagerReportsClient	Fetch reports and price changes
payment	order	OrderServiceClient	Confirm payment on order after verify
shipping	order	OrderServiceClient	Get order details for shipment creation
worker	authentication	ConnectionInterface	Verify worker identity

Connection Diagram

```

authentication → customer, manager, worker
customer → authentication, product order
→ product, manager owner → manager
payment → order shipping → order worker →
authentication notification  RabbitMQ
(async, no Feign)
  
```

3. Request-to-Response Flow

How a request travels through the system end-to-end.

Step	Description
Client	POST /order/create {items, customization, addressId} + JWT header
order	JWTAuthenticationFilter validates JWT using public key
order	OrdersController.createOrder()
order	OrdersService extracts customerId from JWT
order	Calls ProductServiceClient → PUT /product/variants/decrement-stock/{variantId}
product	ProductVariantsService decrements stock, updates inventory
order	Saves Orders record with status PENDING
order	Calls ManagerServiceClient → POST /internal/order-flow/create

manager	InternalManagerController creates ManagerOrderFlow entry
---------	--

3.1 Customer Registration Flow

Step	Description
1. Client	POST /auth/register {name, email, password, role}
2. authentication	AuthController.register() receives request
3. authentication	AuthService validates input, hashes password, saves to Users table
4. authentication	Sends welcome email event to RabbitMQ
5. authentication	Calls ConnectionInterfaceForCustomer → POST /internal/saveNewUser
6. customer	InternalCustomerController.saveNewUser() creates Customers record
7. authentication	JWTService generates access token + refresh token
8. Client	Returns LoginResponseDTO {accessToken, refreshToken, user}
9. notification	RabbitMQ consumer picks up event → EmailService sends welcome email

3.2 Customer Login Flow

Step	Description
1. Client	POST /auth/login {email, password}
2. authentication	AuthController.login()
3. authentication	AuthService loads user from UserRepo, validates password with BCrypt
4. authentication	JWTService.generateToken() creates JWT signed with RSA private key
5. authentication	RefreshTokenService creates refresh token in DB
6. Client	Returns {accessToken, refreshToken, userId, role}

3.3 Place Order Flow

Step	Description
10. Client	Returns OrderResponseDTO {orderId, status, total, items}

3.4 Payment Flow

Step	Description
1. Client	POST /payment/create-order {orderId, amount} + JWT
2. payment	PaymentController.createOrder()
3. payment	PaymentService calls Razorpay API to create payment order
4. Client	Returns {razorpayOrderId, amount, currency} — client opens Razorpay checkout
5. Client	User pays → Razorpay calls client with {paymentId, signature}
6. Client	POST /payment/verify {orderId, paymentId, signature}
7. payment	PaymentService verifies HMAC signature with Razorpay secret
8. payment	Saves PaymentRecord with status SUCCESS
9. payment	Calls OrderServiceClient → PUT /order/internal/payment/confirm/{orderId}
10. order	OrdersService updates order status to CONFIRMED
11. Client	Returns {success: true, orderId}

3.5 Add to Cart Flow

Step	Description
1. Client	POST /cart/add {customerId, productId, variantId, quantity} + JWT
2. product	JWTAuthenticationFilter validates JWT
3. product	CartController.addToCart()
4. product	CartService checks if cart exists for customer, creates if not
5. product	Checks if same variant already in cart — if yes increments qty
6. product	Saves CartItem record
7. Client	Returns CartResponseDTO {cartId, items[], total}

3.6 Manager Order Flow (Production Lifecycle)

Step	Description
1. Manager	PUT /manager/order-flow/start/{orderId} + JWT
2. manager	OrderFlowController.startOrder()
3. manager	OrderFlowService updates ManagerOrderFlow status to IN_PRODUCTION
4. Manager	PUT /manager/order-flow/complete/{orderId}
5. manager	Status → PRODUCTION_COMPLETE
6. Manager	PUT /manager/order-flow/quality-check/{orderId}

Step	Description
7. manager	Status → QUALITY_CHECK

8. Manager	PUT /manager/order-flow/ready-shipping/{orderId}
9. manager	Status → READY_FOR_SHIPPING
10. Manager	PUT /manager/order-flow/ship/{orderId}
11. manager	Status → SHIPPED, triggers shipping service

4. Service Deep Dives

4.1 Authentication Service

Port	8081	Database	gajraj_paithani_auth
------	------	----------	----------------------

Models / Entities

- Users (id, email, password, role, createdAt)
- RefreshToken (id, token, userId, expiryDate)

DTOs

AuthLoginDTO, LoginResponseDTO, RegisterRequestDTO, WorkerRegisterDTO, GoogleOAuthRequestDTO, FacebookOAuthRequestDTO, RequestRefreshTokenDTO, ResponseRefreshTokenDTO, UserResponseDTO, UpdateUserDTO, UpdateCustomerDTO, UpdateManagerDTO, UpdateWorkerDTO, UpdateUserInfoDTO

Services

- AuthService — registration, login, update, delete
- OAuthService — Google & Facebook token verification
- JWTService — generate/validate JWT with RSA keys
- RefreshTokenService — create/validate/rotate refresh tokens

API Endpoints

Method	Path	Description
POST	/auth/register	Register customer
POST	/auth/login	Login with email/password
POST	/auth/admin/login	Login for manager/owner/worker
POST	/auth/oauth/google	Google OAuth login
POST	/auth/oauth/facebook	Facebook OAuth login
POST	/auth/refresh	Refresh access token
POST	/auth/admin/refresh	Refresh admin token
GET	/auth/getUserInfo	Get current user info
PUT	/auth/updateUser/{userId}/{callId}	Update user
DELETE	/auth/deleteUser/{userId}	Delete user

Feign Connections

OUT: → customer /internal/saveNewUser

OUT: → manager /internal/saveNewUser

OUT: → worker /internal/saveNewUser

IN: customer (get user info)

IN: worker (verify identity)

Notes

Uses RSA key pair (private.pem to sign, public.pem to verify). Public key shared with all services.

4.2 Customer Service

Port	8082	Database	gajraj_paithani_customers
------	------	----------	---------------------------

Models / Entities

- Customers (id, userId, name, email, phone, profileImage)
- Addresses (id, customerId, street, city, state, pincode, isDefault)

DTOs

SaveUserReq, CustomerProfileUpdateRequest, CustomerUpdateInDataBase, AddressSaveRequestDTO

Services

- CustomerService — profile, address management
- InternalCustomerService — internal CRUD called by auth
- JWTService — JWT validation

API Endpoints

Method	Path	Description
GET	/customer/profile	Get customer profile
POST	/customer/address	Add new address
GET	/customer/address	Get all addresses
POST	/internal/saveNewUser	Create customer on register (called by auth)
PUT	/internal/updateCustomer/{id}	Update customer
DELETE	/internal/customers/{id}	Delete customer
GET	/internal/customers/profile/{userId}	Get profile by userId
GET	/internal/customers	Get all customers
GET	/internal/customers/{id}	Get customer by id
POST	/internal/customers/{id}/address	Add address (internal)
PUT	/internal/customers/{id}/address/{addrId}	Update address (internal)
DELETE	/internal/customers/{id}/address/{addrId}	Delete address (internal)

Feign Connections

OUT: → authentication (get user info)

OUT: → product (product data)

IN: authentication (saveNewUser, update, delete)

Notes

Stores customer-specific data separately from auth Users table. Linked by userId.

4.3 Product Service

Port	8087	Database	gajraj_paithani_products + AWS S3
------	------	----------	-----------------------------------

Models / Entities

- Products

- ProductVariants
- ProductCategories
- ProductImages
- ProductAttributes
- Inventory
- InventoryAuditLog
- StockHistory
- Cart
- CartItem
- Wishlist
- ConfigAsset
- BodyColor
- Border
- BorderColor
- Butti
- Padar

DTOs

ProductCreateRequestDTO, ProductDetailDTO, ProductListResponseDTO, ProductUpdateRequestDTO, VariantResponseDTO, VariantUpdateDTO, CategoryRequestDTO, CategoryResponseDTO, ImageResponseDTO, InventoryItemDTO, InventoryResponseDTO, InventoryUpdateDTO, StockUpdateDTO, StockHistoryDTO, CartResponseDTO, CartSummaryResponseDTO, AddToCartRequestDTO, UpdateCartItemRequestDTO, WishlistResponseDTO, ConfigAssetCreateDTO, ConfigAssetResponseDTO, BodyColorResponseDTO, BorderResponseDTO, BorderColorResponseDTO, ButtiResponseDTO, PadarResponseDTO

Services

- ProductsService
- ProductVariantsService
- ProductCategoriesService
- ProductImagesService
- S3Service — AWS S3 presigned URLs
- InventoryService
- StockManagementService
- CartService
- WishlistService
- ConfigAssetService• BodyColorService
- BorderColorService
- BorderService
- ButtiService
- PadarService

API Endpoints

Method	Path	Description
--------	------	-------------

GET	/product/all	List all products
POST	/product/create	Create product
GET	/product/{productId}	Get product
GET	/product/detail/{productId}	Get product with full details
PUT	/product/update/{productId}	Update product
PUT	/product/archive/{productId}	Archive product
GET	/product/categories	List categories
POST	/product/categories/create	Create category
GET	/product/upload-url	Get S3 presigned upload URL
POST	/cart/add	Add item to cart
GET	/cart/{customerId}	Get cart
GET	/cart/{customerId}/summary	Get cart summary with totals
PATCH	/cart/item/{cartItemId}	Update cart item qty
DELETE	/cart/item/{cartItemId}	Remove cart item
DELETE	/cart/clear/{customerId}	Clear cart
POST	/wishlist/add	Add to wishlist
GET	/wishlist/{customerId}	Get wishlist
DELETE	/wishlist/remove/{id}	Remove from wishlist
GET	/product/inventory/all	Get inventory
PUT	/product/inventory/update/{variantId}	Update inventory
GET	/product/inventory/low-stock	Get low stock items

Feign Connections

IN: order (decrement stock)

IN: customer (product data)

Notes

Largest service. Handles textile-specific attributes (Butti=motif, Padar=border design, Border=border type). S3 for all images.

4.4 Order Service

Port	8083	Database	gajraj_paithani_orders
------	------	----------	------------------------

Models / Entities

- Orders (id, customerId, status, total, paymentStatus, createdAt)
- Customization (id, orderId, variantId, notes)

DTOs

OrderCreateRequestDTO, OrderResponseDTO, OrderListResponseDTO

Services

- OrdersService — full order lifecycle

- JWTService — JWT validation

API Endpoints

Method	Path	Description
POST	/order/create	Create new order
GET	/order/my-orders	Get customer's orders
GET	/order/all	Get all orders (manager)
GET	/order/{orderId}	Get order details
PUT	/order/cancel/{orderId}	Cancel order
PUT	/order/status/{orderId}	Update order status
PUT	/order/internal/payment/confirm/{orderId}	Confirm payment (called by payment service)

Feign Connections

OUT: → product (decrement stock on create)

OUT: → manager (create order flow) **IN:**

payment (confirm payment)

IN: shipping (get order for shipment)

Notes

Central to the system. Connected to product, manager, payment and shipping.

4.5 Manager Service

Port	8085	Database	gajraj_paithani_manager
------	------	----------	-------------------------

Models / Entities

- Managers
- ManagerOrderFlow
- ManagerOrderAssignments• OrderHoldCases
- WorkerPerformanceActions
- WorkerVerificationDecision
- CustomerSupportCase
- ProductPriceUpdates
- OwnerReports

DTOs

OrderFlowResponseDTO, OrderFlowListDTO, ReportCreateDTO, ReportResponseDTO, ReportListResponseDTO, SupportCaseCreateDTO, SupportCaseResponseDTO, SupportCaseListDTO, PriceChangeCreateDTO, PriceChangeResponseDTO, PriceChangeListResponseDTO, SaveUserDTO

Services

- ManagerService
- OrderFlowService
- ReportsService

- SupportCaseService
- PriceChangeService
- InternalManagerService

API Endpoints

Method	Path	Description
GET	/manager/order-flow/all	List all order flows
GET	/manager/order-flow/{orderId}	Get order flow
PUT	/manager/order-flow/start/{orderId}	Start production
PUT	/manager/order-flow/complete/{orderId}	Complete production
PUT	/manager/order-flow/quality-check/{orderId}	Quality check
PUT	/manager/order-flow/ready-shipping/{orderId}	Ready for shipping
PUT	/manager/order-flow/ship/{orderId}	Mark shipped
GET	/manager/reports/all	List reports
POST	/manager/reports/create	Create report for owner
GET	/manager/support/all	List support cases
POST	/manager/support/create	Create support case
GET	/manager/support/stats	Support statistics
GET	/manager/price-changes/all	List price change requests
POST	/manager/price-changes/create	Create price change request
POST	/internal/saveNewUser	Save manager (called by auth)
POST	/internal/order-flow/create	Create order flow (called by order)
GET	/internal/order-flow/status/{orderId}	Get order flow status

Feign Connections

OUT: → customer (get customer info for support cases)

IN: authentication (saveNewUser)

IN: order (createOrderFlow)

IN: owner (reports/prices)

Notes

Hub for all business operations. Manages full production lifecycle of orders.

4.6 Worker Service

Port	8084	Database	gajraj_paithani_workers
------	------	----------	-------------------------

Models / Entities

- Workers
- WorkersAssignment
- WorkerPerformance
- WorkerProgress
- WorkerVerification

- Materials
- MaterialUsageForOrder

DTOs

SaveUserReq, UpdateWorkerProfileRequest, UpdateWorkerInDataBase

Services

- WorkerService — worker profile
- InternalWorkerService — internal CRUD
- ManagerOperationOnWorkers — manager actions on workers
- JWTService

API Endpoints

Method	Path	Description
GET	/getWorker	Get worker profile
POST	/internal/saveNewUser	Save worker on register
DELETE	/internal/deleteWorker/{userId}	Delete worker
POST	/internal/updateWorker/{workerId}	Update worker

Feign Connections

OUT: → authentication (verify worker identity)

IN: authentication (saveNewUser, delete, update)

Notes

Tracks worker materials usage, performance metrics, and verification status.

4.7 Payment Service

Port	8088	Database	gajraj_paithani_payments
------	------	----------	--------------------------

Models / Entities

- PaymentRecord (id, orderId, razorpayOrderId, razorpayPaymentId, amount, status)

DTOs

CreateOrderRequest, CreateOrderResponse, VerifyPaymentRequest

Services

- PaymentService — Razorpay integration
- JWTService

API Endpoints

Method	Path	Description
POST	/payment/create-order	Create Razorpay order
POST	/payment/verify	Verify payment signature

Feign Connections

OUT: → order /order/internal/payment/confirm/{orderId}

Notes

Uses Razorpay. Verifies HMAC-SHA256 signature before confirming payment.

4.8 Shipping Service

Port	8089	Database	gajraj_paithani_shipping
------	------	----------	--------------------------

Models / Entities

- Shipment
- ShipmentTracking
- ShippingProviderConfig

DTOs

CreateShipmentRequest, ShipmentResponse, TrackingResponse, TrackingEventResponse, CancelShipmentRequest, NextStatusRequest

Services

- ShippingService
- JWTService

API Endpoints

Method	Path	Description
POST	/shipping/create	Create shipment
GET	/shipping/{orderId}	Get shipment by order
GET	/shipping/tracking/{trackingNumber}	Track shipment
POST	/shipping/cancel	Cancel shipment
POST	/shipping/mock/next-status	Advance mock status (for testing)

Feign Connections

OUT: → order (get order details)

Notes

Uses MockShippingProvider — pluggable provider pattern for real courier integration later.

4.9 Owner Service

Port	8086	Database	gajraj_paithani_owner
------	------	----------	-----------------------

DTOs

OwnerDashboardDTO

Services

- OwnerService — dashboard, report management
- JWTService

API Endpoints

Method	Path	Description
GET	/owner/dashboard	Get owner dashboard data

GET	/owner/reports	List all reports from managers
GET	/owner/reports/{reportId}	Get report details
PUT	/owner/reports/approve/{reportId}	Approve report
PUT	/owner/reports/reject/{reportId}	Reject report
GET	/owner/price-changes	List price change requests
PUT	/owner/price-changes/approve/{id}	Approve price change
PUT	/owner/price-changes/reject/{id}	Reject price change

Feign Connections

OUT: → manager (get reports and price changes)

Notes

Top-level role. Can only approve/reject — no direct data mutations.

4.10 Notification Service

Port	4000	Database	None
------	------	----------	------

Services

- EmailService — nodemailer SMTP
- email.consumer — RabbitMQ consumer

Notes

Listens on RabbitMQ mail_queue. Sends emails using nodemailer. Currently handles welcome emails from auth service.

5. Frontend Services

5.1 Customer Frontend (Next.js)

Port: 3000 | Stack: Next.js 15, TypeScript, Zustand, NextAuth

Page/Route	File	Description
/	app/page.tsx	Landing page with 3D scene
/login	(auth)/login/page.tsx	Customer login
/register	(auth)/register/page.tsx	Customer registration
/collections/[category]	(main)/collections/[category]/page.tsx	Browse products by category
/product/detail/[id]	(main)/product/detail/[id]/page.tsx	Product detail with variants
/cart	(main)/cart/page.tsx	Shopping cart
/checkout	(main)/checkout/page.tsx	Checkout
/payment	(main)/payment/page.tsx	Razorpay payment
/orders	(main)/orders/page.tsx	Order history
/orders/[orderId]	(main)/orders/[orderId]/page.tsx	Order detail & tracking
/wishlist	(main)/wishlist/page.tsx	Wishlist
/profile	(main)/profile/page.tsx	Customer profile

/help	(main)/help/page.tsx	Help & support
-------	----------------------	----------------

State Management (Zustand Stores)

- cartStore — cart items and count
- customerStore — customer profile
- addressStore — customer addresses
- notificationStore — toast notifications
- menuStore — sidebar/menu state
- helloStore — misc

5.2 Manager Frontend (Next.js)

Port: 3002 | Stack: Next.js 15, TypeScript, TanStack Query, Zustand

Section	Pages	Description
Dashboard	dashboard, order-stats, active-workers, pending-actions, quick	links, recent-orders, revenueOverview of all business metrics
Orders	orders, assign, holds, progress, quality, shipping	Full order lifecycle management
Products	add, edit, archived, categories, images, preview, variants, ass	tsProduct catalogue management
Inventory	inventory, low-stock, update, history	Stock management
Workers	workers, actions, performance, verification, workload	Worker management
Customers	customers, [customerId]	Customer overview

Section	Pages	Description
Reports	reports, create, price-requests	Reports to owner
Support	support, linked-orders, stats	Customer support cases
Settings	settings, theme	App settings

6. Infrastructure & Configuration

6.1 Port Map

Service	Port	Protocol
server_registry (Eureka)	8761	HTTP
authentication	8081	HTTP + JWT
customer	8082	HTTP + JWT
order	8083	HTTP + JWT
worker	8084	HTTP + JWT
manager	8085	HTTP + JWT
owner	8086	HTTP + JWT
product	8087	HTTP + JWT
payment	8088	HTTP + JWT
shipping	8089	HTTP + JWT
notification	4000	RabbitMQ consumer
RabbitMQ	5672 / 15672	AMQP / Management UI
PostgreSQL	5432	TCP
customer-frontend	3000	HTTP
manager-frontend	3002	HTTP

6.2 Environment Variables Needed

Service	Variable	Description
authentication	FACEBOOK_APP_ID	Facebook OAuth App ID
authentication	FACEBOOK_APP_SECRET	Facebook OAuth App Secret
product	AWS_ACCESS_KEY_ID	AWS access key for S3
product	AWS_SECRET_ACCESS_KEY	AWS secret key for S3
payment	razorpay.key.id	Razorpay Key ID (in properties)
payment	razorpay.key.secret	Razorpay Key Secret (in properties)
notification	EMAIL_USER	SMTP email address
notification	EMAIL_PASS	SMTP email password
notification	RABBITMQ_URL	RabbitMQ connection URL
All Spring services	DB password	PostgreSQL password (currently hardcoded as root)

6.3 Remaining Deployment Tasks

Task	Status	Notes
Real Dockerfiles for all Spring Boot services	Pending	Maven multi-stage build
Real Dockerfile for notification (Node.js)	Pending	Node + npm install

Real Dockerfiles for frontends (Next.js)	Pending	Next.js multi-stage build
Update application.properties for Docker	Pending	localhost → container names
docker-compose.yml with full config	Pending	ports, env vars, depends_on, networks
PostgreSQL container in compose	Pending	One DB per service or shared
RabbitMQ container in compose	Pending	notification + auth need this
Envoy gateway config (envoy.yaml)	Pending	Path-based routing to services
GitHub Actions push to ECR	Pending	After local testing works
EC2 deployment	Pending	Final step